MONITOR Z · L01.2

Clase 2 — Los registros y las instrucciones de todos los días

Cuaderno de estudio · versión del 2026-10-28

Margen derecho y renglones libres para tus notas. Las láminas son las mismas de la carpeta **_visual**. Si una lección se corrige, se regenera este cuaderno con la fecha nueva.

L01.2 — Clase 2 · Los registros y las instrucciones de todos los días

Temario de la L01: 1. La forma de un programa · **2. Los registros y las instrucciones de todos los días** · 3. Entrar y salir: prólogo, save area, epílogo y RC · 4. Los datos: DS, DC, literales, el PARM y el `EX` · 5. Hablar con el sistema y leer el listado: macros, SVC, DCB, AMODE/RMODE.

Objetivo de esta clase: leer una instrucción de ZTKHOLA **byte por byte** y saber qué hace. Al terminar tenés que poder:

- decir para qué usa cada registro tu programa, y cuáles no se tocan;
- decodificar a mano un `Object Code` del listado, como `5010 C0CC`;
- saber cuántos bytes ocupa una instrucción mirando sólo su primer byte;
- seguir un salto: qué pone el **código de condición** y qué mira el `BC`.

> 📚 Fuentes: - `monitor_z/asm/ZTKHOLA.asm`: los números de **renglón**. - El listado de **JOB24962** (2026-10-07 10:41, DESA): los números de **sentencia** y los bytes. - JESMSGLG y JESJCL de JOB24962: `RUN1 04` con `PARM='PRUEBA'`, `RUN2 08` sin PARM, y el SYSPRINT de RUN1: `+HOLA PARM=PRUEBA`. - El listado de **JOB34775** (ZTKMON v0.9): sentencias 10126–10132 y el *Using Map* (página 227). - *z/Architecture Principles of Operation* (SA22-7832), cap. 5 (formatos de instrucción) y cap. 7 (instrucciones generales).

1. Los 16 registros: los bolsillos de la CPU

Analogía: la memoria es un **depósito enorme** y los registros son los **16 bolsillos** del que trabaja. Casi todo lo que hace ZTKHOLA sigue el mismo camino: traer el dato al bolsillo, trabajarlo ahí y, si hace falta, devolverlo. (Hay instrucciones que trabajan directo en el depósito, como `MVC`, que copia de memoria a memoria; las vemos en la sección 4.)

Los registros generales van de **R0 a R15**. En el hardware de hoy miden 64 bits, pero las instrucciones de esta clase (`L`, `ST`, `LR`, `LTR`…) trabajan con la **mitad derecha, 32 bits**: los 4 bytes de un fullword (`F`).

En Assembler los 16 son «iguales» para la CPU, pero **no para z/OS**. Hay una **convención** de enlace que respetan todos los programas, y ZTKHOLA la cumple al pie de la letra:

Registro	Al **entrar** al programa trae…	Al **salir** tiene que llevar…	Dónde lo ves en ZTKHOLA
R0	(nada que importe)	(nada que importe)	`PUT` lo usa: `LA 0,SALIDA` (sent. 80)
R1	la dirección de la **lista del PARM**	(nada que importe)	`ST 1,SALVAR1` (renglón 15) la guarda antes de que `OPEN` la pise
R2–R11	(los del que llamó)	**los mismos** que trajo	ZTKHOLA usa R2 y R3; el `LM` del final los repone
R12	(el del que llamó)	el mismo	**tu base**, por elección (sección 2)
R13	la dirección de la **save area del que llamó**	la misma	`ST 13,4(,2)` y `L 13,4(,13)` (clase 3)
R14	la **dirección de retorno**	la misma	`BR 14` (renglón 78) vuelve ahí
R15	la dirección de **entrada** del programa	el **return code**	`LR 12,15` (renglón 13) y `L 15,RCFINAL` (renglón 73)

El testigo de que la convención importa está en tu comentario del renglón 74: *«PUT Y CLOSE PISAN R15»*. El listado lo muestra. La macro `PUT` se expandió así:

```
00009A 4110 C22C      0022C    79+         LA    1,PRTDCB          LOAD I
00009E 4100 C11C      0011C    80+         LA    0,SALIDA          LOAD I
0000A2 1FFF                    81+         SLR   15,15             CLEAR
0000A4 BFF7 1031      00031    82+         ICM   15,7,49(1)        LOAD I
0000A8 0DEF                    83+         BASR  14,15             LINK T
```

`PUT` usa **R0, R1, R14 y R15**, los cuatro registros «de trabajo» de la convención. Si hubieras cargado el RC en R15 antes del `PUT`, se habría perdido. Por eso el RC vive en memoria (`RCFINAL`) y recién se carga en R15 en el epílogo.

> 💡 Regla práctica: **R0, R1, R14 y R15 no sobreviven a una macro.** R2 a R12 sí, porque la rutina llamada los guarda y los repone (clase 3).

Los 16 registros: iguales para la CPU, no para z/OS

L01 · Clase 2 · Los registros y las instrucciones



Registro	Al entrar trae…	Al salir lleva…	En ZTKHOLA
R1	la dirección de la lista del PARM	—	`ST 1,SALVAR1` (renglón 15): se guarda antes de que OPEN la pise
R2–R12	los del que llamó	**los mismos**	usás R2, R3 y R12; el `LM 0,12,20(13)` los repone
R13	la save area del que llamó	la misma	clase 3
R14	la dirección de retorno	la misma	`BR 14` (renglón 78)
R15	la dirección de entrada	**el return code**	`LR 12,15` (13) · `L 15,RCFINAL` (73)

⚠️ **R0, R1, R14 y R15 no sobreviven a una macro.** La expansión de `PUT` (sentencias 79–83) usa los cuatro: `LA 1`, `LA 0`, `SLR 15,15`, `ICM 15`, `BASR 14,15`. Por eso el RC vive en `RCFINAL` y recién va a R15 en el epílogo.

Fuente: ZTKHOLA.asm (renglones 13, 15, 73, 74, 78) · listado de JOB24962 (sentencias 79–83) · z/OS MVS Assembler Services Guide, linkage conventions

Lámina L01-14_los_16_registros

2. Base + desplazamiento: cómo encuentra la CPU tus datos

Una instrucción **no lleva la dirección** del dato. Lleva **un registro base y una distancia** (el desplazamiento, de 12 bits: de 0 a 4.095). La CPU suma las dos cosas en el momento de ejecutar.

Analogía: «el tercer cajón **desde la puerta**». No hace falta saber en qué calle está el edificio: alcanza con saber dónde está la puerta (R12) y contar.

Por eso el prólogo hace esto:

Renglón	Sentencia	Qué hace
13	`LR 12,15`	R15 trae la dirección donde z/OS cargó ZTKHOLA. La copia en R12.
14	`USING ZTKHOLA,12`	Le **promete** al ensamblador: «en ejecución, R12 va a valer la dirección de `ZTKHOLA`».

Con esa promesa, cada vez que escribís un nombre (`SALVAR1`), el ensamblador calcula «está a `X'0CC'` del principio» y arma la instrucción con **base 12, desplazamiento `0CC`**. Así quedó tu renglón 15:

```
000006 5010 C0CC      000CC    15          ST    1,SALVAR1
```

Byte(s)	Valor	Quiere decir
1.º	`50`	código de operación: **ST** (guardar un fullword)
2.º, izquierda	`1`	R1: el registro que se guarda
2.º, derecha	`0`	sin registro índice
3.º, izquierda	`C`	**base = R12**
3.º derecha + 4.º	`0CC`	**desplazamiento = X'0CC'** = 204 → `SALVAR1` (su `Loc` es `0000CC`)

En ejecución la CPU calcula: **dirección = contenido de R12 + X'0CC'**. Si mañana z/OS carga ZTKHOLA en otro lugar, R12 cambia y la cuenta sigue dando bien. Por eso el mismo load module corre en cualquier dirección.

El listado te lo confirma arriba de cada página: `Active Usings: ZTKHOLA,R12`.

El límite de 4.095. Un registro base alcanza **4 KB**. ZTKHOLA mide 681 bytes y le sobra con R12. ZTKMON no:

```
000004 18CF                    10127          LR    R12,R15
000006 41B0 CFFF         00FFF 10128          LA    R11,4095(,R12)
00000A 41B0 B001         00001 10129          LA    R11,1(,R11)
00000E 41A0 BFFF         00FFF 10130          LA    R10,4095(,R11)
000012 41A0 A001         00001 10131          LA    R10,1(,R10)
                               10132          USING ZTKMON,R12,R11,R1
```

R11 = R12 + 4.096 y R10 = R11 + 4.096: **tres bases**, que cubren de `X'0000'` a `X'2FFF'` (12 KB). El *Using Map* del final del listado mide cuánto se estiró cada una: R12 hasta el desplazamiento `FF2`, R11 hasta `FF0` y R10 hasta `873`. Las dos primeras están **casi llenas**.

> ⚠️ ZTKMON mide 63.276 bytes y las tres bases cubren 12.288. Cómo alcanza lo que está más allá de `X'3000'` **no lo medí en esta clase**: queda para cuando leamos ZTKMON por dentro.

> 🔁 Un caso que no usa base: el `WTO` se expandió con `BRAS 1,IHB0003A` (`A715 0015`, sent. 39). Es un salto **relativo**: lleva «saltá 21 medias palabras (X'15' × 2 = 42 bytes) desde acá», no una base. `X'34' + X'2A' = X'5E'`, el `Loc` de `IHB0003A`.



Lámina L01-15_base_y_desplazamiento

3. El primer byte te dice el largo

Las instrucciones miden **2, 4 o 6 bytes**, nunca otra cosa. Y no hace falta memorizar cuál mide cuánto: lo dicen los **dos primeros bits** del código de operación.

Primeros 2 bits	Primer byte	Largo	Formato típico	Tus instrucciones
`00`	`00`–`3F`	**2 bytes**	RR: registro con registro	`LR` `18`, `LTR` `12`, `BCTR` `06`, `BR` `07`, `SVC` `0A`, `BASR` `0D`, `SLR` `1F`
`01`	`40`–`7F`	**4 bytes**	RX: registro con memoria	`LA` `41`, `EX` `44`, `BC` `47`, `LH` `48`, `CH` `49`, `ST` `50`, `L` `58`
`10`	`80`–`BF`	**4 bytes**	RS / SI / RI	`STM` `90`, `TM` `91`, `LM` `98`, `BRAS` `A7`, `ICM` `BF`
`11`	`C0`–`FF`	**6 bytes**	SS: memoria con memoria	`MVC` `D2`

Tus 31 instrucciones, por largo:

Largo	Cuántas	Bytes	Cuáles
2	5	10	`LR` ×2, `LTR`, `BCTR`, `BR`

4	20	80	`L` ×5, `ST` ×3, `LH` ×2, saltos ×4, `STM`, `LM`, `LA`, `TM`, `CH`, `EX`
6	6	36	`MVC` ×6 (una es `COPIAPRM`, que está entre los datos)
	31	**126**	

Y cierra con lo que mediste en la clase 1: el código ocupa 204 bytes. **120** son tus 30 instrucciones del código (126 menos los 6 de `COPIAPRM`), y los otros **84** los trajeron las macros: `OPEN` 12, `WTO` 46, `PUT` 16 y `CLOSE` 10.

> ✅ Esto también responde la pregunta 3 de la clase 1. El bloque 2 (renglones 40–44) son 5 instrucciones: `L` 4, `L` 4, `LH` 4, `LTR` **2** y `BZ` 4. Son 18 bytes, del `Loc 00006A` al `00007B`.

El primer byte te dice cuánto mide la instrucción

L01 · Clase 2 · Los registros y las instrucciones

2 primeros bits	Primer byte	Largo	Formato	Tus instrucciones (código de operación)
`00`	`00–3F`	**2**	RR · registro con registro	`LR 18 · LTR 12 · BCTR 06 · BR 07 · SVC 0A`
`01`	`40–7F`	**4**	RX · registro con memoria	`LA 41 · EX 44 · BC 47 · LH 48 · CH 49 · ST 50 · L 58`
`10`	`80–BF`	**4**	RS · SI · RI	`STM 90 · TM 91 · LM 98 · BRAS A7 · ICM BF`
`11`	`C0–FF`	**6**	SS · memoria con memoria	`MVC D2`

Tus 31 instrucciones

5 de 2 bytes + 20 de 4 + 6 de 6 = **126 bytes**. Sin `COPIAPRM` (está entre los datos): **120**.

Lo que pusieron las macros

`OPEN` 12 · `WTO` 46 · `PUT` 16 · `CLOSE` 10 = **84**. 120 + 84 = los **204** bytes de código de la clase 1.

Pregunta 3 de la clase 1

Bloque 2: `L` 4 · `L` 4 · `LH` 4 · `LTR` **2** · `BZ` 4 = 18 bytes, de `00006A` a `00007B`.

Familia	Instrucciones	Lo que hay que recordar
traer y llevar	`L ST LH LR LM STM LA`	`LA` trae la **dirección**, no el contenido
preguntar	`LTR CH TM`	no cambian registros: prenden el código de condición
saltar	`B BO BZ BNH BR`	todas son `BC` / `BCR` con otra máscara
mover y contar	`MVC BCTR SLR`	el largo de `MVC` va guardado **menos 1**: `D284` = 133

Fuente: listado de JOB24962 (columnas Loc y Object Code, sentencias 12–121) · Principles of Operation, cap. 5 «Instruction formats»

Lámina L01-16_el_primer_byte

4. Las instrucciones de todos los días

Con estas cuatro familias se escribe casi todo ZTKHOLA (y la mayor parte del cuerpo de ZTKMON):

4.1 🎒 Traer y llevar: memoria ↔ registro

Instrucción	Qué hace	En ZTKHOLA	Bytes
`L r,dato`	**Load**: trae 4 bytes de memoria al registro	`L 2,SALVAR1` (40)	`5820 C0CC`
`ST r,dato`	**Store**: lleva 4 bytes del registro a memoria	`ST 1,SALVAR1` (15)	`5010 C0CC`
`LH r,dato`	**Load Halfword**: trae **2** bytes (con signo)	`LH 3,0(,2)` (42): la longitud del PARM	`4830 2000`
`LR r1,r2`	copia un registro en otro	`LR 12,15` (13)	`18CF`
`LM / STM`	trae / lleva **varios** registros seguidos	`STM 14,12,12(13)` (12)	`90EC D00C`
`LA r,dato`	**Load Address**: pone en el registro **la dirección**, sin leer la memoria	`LA 2,SAVEAREA` (16)	`4120 C0D4`

> ⚠️ **`L` y `LA` se confunden y no son lo mismo.** `L 2,SAVEAREA` traería los 4 bytes que hay **dentro** de la save area. `LA 2,SAVEAREA` pone en R2 **dónde está** la save area. Para encadenarla (clase 3) necesitás la dirección: `LA`.

`STM 14,12,12(13)` byte por byte (`90EC D00C`): `90` = STM, `E` = desde R14, `C` = hasta R12, `D` = base R13, `00C` = desplazamiento 12. Como «da la vuelta» (R14, R15, R0, R1 … R12), guarda **15 registros**: todos menos R13, que se guarda aparte. Son 60 bytes, a partir del byte 12 de la save area del que llamó.

4.2 🔍 Preguntar: comparar y probar

Estas instrucciones **no cambian ningún registro**. Sólo dejan una respuesta en el **código de condición** (sección 5):

Instrucción	Qué pregunta	En ZTKHOLA	Bytes
`LTR r,r`	¿el registro es cero, negativo o positivo?	`LTR 3,3` (43): ¿vino PARM?	`1233`
`CH r,dato`	¿el registro es igual, menor o mayor que un halfword?	`CH 3,=H'100'` (55)	`4930 C29C`
`TM dato,máscara`	¿qué bits de este byte están prendidos?	`TM PRTDCB+48,X'10'` (28): ¿abrió?	`9110 C25C`

En `TM` la máscara (`10`) va **adentro** de la instrucción: `91` `10` `C25C`. Y `C25C` es base R12 + `25C`, que es `PRTDCB+48` (`22C + 30 = 25C`): el byte `OFLGS` del DCB, que en el listado arranca en `02` (sent. 144) y `OPEN` cambia.

4.3 🔁 Saltar

Instrucción	Qué es en realidad	Bytes	Salta si…
`B SALIR` (32)	`BC 15,SALIR`	`47F0 C0BA`	siempre
`BO ABRIO` (29)	`BC 1,ABRIO`	`4710 C06A`	los bits probados están **todos en 1**
`BZ SINPARM` (44)	`BC 8,SINPARM`	`4780 C0B0`	el resultado fue **cero**
`BNH LARGOOK` (56)	`BC 13,LARGOOK`	`47D0 C094`	**no** fue mayor
`BR 14` (78)	`BCR 15,14`	`07FE`	siempre, a la dirección que hay en R14

Todos son **la misma instrucción**: `BC` (*Branch on Condition*, `47`) o su versión de registro, `BCR` (`07`). Lo único que cambia es el **segundo byte**, la máscara: `F`, `1`, `8`, `D`. `BO`, `BZ` y `BNH` son apodos que pone el ensamblador (*mnemónicos extendidos*) para que no tengas que acordarte de los números.

4.4 📦 Mover memoria y hacer cuentas

Instrucción	Qué hace	En ZTKHOLA	Bytes
`MVC destino,origen`	copia de 1 a 256 bytes de memoria a memoria	`MVC SALIDA,ESPACIOS` (53)	`D284 C11C C1A1`
`BCTR r,0`	le resta 1 al registro (con `0`, no salta)	`BCTR 3,0` (59)	`0630`
`SLR r,r`	resta un registro de sí mismo = lo pone en cero	`SLR 15,15` (de `PUT`)	`1FFF`

El largo de `MVC` va guardado menos 1, y está en el listado:

Renglón	Sentencia	Segundo byte	Bytes que copia
20	`MVC RCFINAL,=F'8'`	`03`	4
54	`MVC SALIDA(11),=C'+HOLA PARM='`	`0A` = 10	11
53	`MVC SALIDA,ESPACIOS`	`84` = 132	133
93	`COPIAPRM MVC SALIDA+11(0),2(2)`	`00`	el que le ponga `EX` (clase 4)

Por eso el `BCTR 3,0` del renglón 59 tiene el comentario *«MVC CUENTA DE 0»*: antes del `EX`, el largo se baja en 1.

5. El código de condición: la respuesta que mira el salto

Analogía: un **semáforo de cuatro luces** al lado de la CPU. Las instrucciones que preguntan (`LTR`, `CH`, `TM`) prenden **una** luz. El `BC` siguiente no mira el dato: **mira el semáforo**.

El código de condición (CC) vale 0, 1, 2 o 3. Qué significa cada valor depende de quién lo puso:

CC	Después de `LTR`	Después de `CH`	Después de `TM`
0	es cero	son iguales	los bits probados están todos en 0
1	es negativo	el registro es **menor**	mezcla de 0 y 1
2	es positivo	el registro es **mayor**	—
3	—	—	los bits probados están todos en 1

La máscara del `BC` tiene un bit por luz: **8** = CC 0, **4** = CC 1, **2** = CC 2, **1** = CC 3. Salta si la luz prendida tiene su bit en la máscara:

- `BZ` = máscara **8** → salta con CC 0.
- `BO` = máscara **1** → salta con CC 3.
- `BNH` = máscara **13** = 8 + 4 + 1 → salta con CC 0, 1 o 3 (todo menos 2, «mayor»).
- `B` = máscara **15** → salta con cualquier luz: siempre.

Tus dos corridas de JOB24962, paso a paso:

Renglón	Instrucción	RUN1, `PARM='PRUEBA'`	RUN2, sin PARM
42	`LH 3,0(,2)`	R3 = 6	R3 = 0
43	`LTR 3,3`	CC **2** (positivo)	CC **0** (cero)
44	`BZ SINPARM`	máscara 8, luz 2 → **sigue**	máscara 8, luz 0 → **salta**
55	`CH 3,=H'100'`	6 < 100 → CC **1**	—
56	`BNH LARGOOK`	máscara 13, luz 1 → **salta** (no toca el 100)	—
59	`BCTR 3,0`	R3 = 5	—
60	`EX 3,COPIAPRM`	copia 5 + 1 = 6 bytes: `PRUEBA`	—
62	`MVC RCFINAL,=F'4'`	RC 4	(queda el 8 del prólogo)
	resultado medido	`RUN1 04` y `+HOLA PARM=PRUEBA` en SYSPRINT	`RUN2 08`

Lo que predijimos leyendo los bytes es **exactamente** lo que dice el JESMSGLG de JOB24962.

El código de condición: la luz que mira el salto

L01 · Clase 2 · Los registros y las instrucciones

🚦 Cuatro luces, una máscara

CC	Bit	LTR	CH	TM
0	8	cero	igual	todos en 0
1	4	negativo	menor	mezcla
2	2	positivo	mayor	—
3	1	—	—	todos en 1

🔁 Los saltos de ZTKHOLA

Apodo	Bytes	Máscara
`BZ`	`4780`	8 → CC 0
`BO`	`4710`	1 → CC 3
`BNH`	`47D0`	13 = 8+4+1
`B` · `BR 14`	`47F0` · `07FE`	15 → siempre

Renglón	Instrucción	RUN1 · PARM='PRUEBA'	RUN2 · sin PARM
42	`LH 3,0(,2)`	R3 = 6	R3 = 0
43–44	`LTR 3,3` · `BZ`	CC 2 → **sigue**	CC 0 → **salta** a SINPARM
55–56	`CH 3,=H'100'` · `BNH`	CC 1 → salta a LARGOOK	—
59–60	`BCTR` · `EX`	R3 = 5 → copia 6 bytes	—
medido	JESMSGLG · SYSPRINT	**RUN1 04** · `+HOLA PARM=PRUEBA`	**RUN2 08**

Fuente: listado de JOB24962 (sentencias 34–35, 46, 56–58, 69–74, 106) · JESMSGLG y RUN1/SYSPRINT de JOB24962 · Principles of Operation, cap. 7 (BC)

Lámina L01-17_codigo_de_condicion

Resumen

Idea	En una línea	Testigo
16 registros	R0–R15; la CPU los trata igual, la convención de z/OS no	renglones 13, 15, 73, 78
R0, R1, R14, R15	se pierden en cada macro: el RC vive en memoria hasta el final	`PUT`, sentencias 79–83
Base + desplazamiento	dirección = registro base + 0…4.095; `USING` le dice al ensamblador cuál base usar	`5010 C0CC`
4 KB por base	ZTKHOLA usa una (R12); ZTKMON tres (R12, R11, R10)	JOB34775, sent. 10127–10132 y *Using Map*
El largo	los 2 primeros bits del código: `00` → 2, `01` / `10` → 4, `11` → 6 bytes	31 instrucciones = 126 bytes
`L` vs `LA`	`L` trae el contenido; `LA` trae la dirección	renglones 40 y 16
`MVC`	el largo va guardado menos 1	`D284` = 133 bytes

CC + `BC`	`LTR` / `CH` / `TM` prenden una luz; `BC` salta según su máscara (8-4-2-1)	RUN1 04, RUN2 08

Preguntas de comprobación

1. Decodificá `58D0 D004` (sentencia 103, renglón 75): qué instrucción es, qué registro carga, cuál es la base y cuál el desplazamiento. En ese momento R13 apunta a tu `SAVEAREA`: ¿qué hay en esos 4 bytes y por qué es lo que hay que cargar?
2. En el renglón 16 está `LA 2,SAVEAREA`. Si alguien lo cambiara por `L 2,SAVEAREA`, ¿qué quedaría en R2? ¿Y qué pasaría en el renglón 17, `ST 13,4(,2)`?
3. Suponé que corrés ZTKHOLA con `PARM='A'` (1 byte). Seguí la tabla de la sección 5: ¿qué CC deja `LTR`? ¿Salta `BNH`? ¿Qué vale R3 cuando llega al `EX`, y cuántos bytes copia? ¿Qué RC esperarías?

Mis respuestas

Relacionado: *L01.1 - Clase 1 - La forma de un programa - renglones, columnas y sentencias* · *L01 - Anatomia de un programa Assembler - el prologo de ZTKMONS* · *P01 - ZTKHOLA* · *Ruta de aprendizaje*.

Notas